一、机器字长与存储字长区别

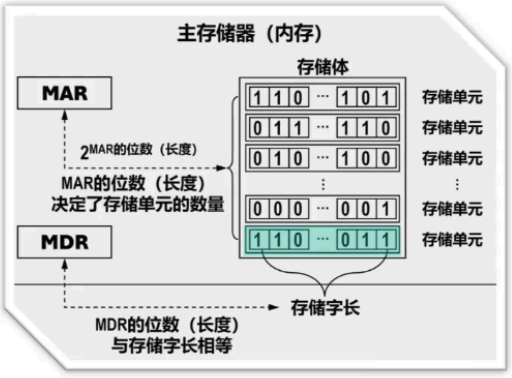
主存中数据的存放 —— 机器字长与存储字长的区别

机器字长 ● CPU一次能够处理的二进制数据的位数

存储字长 ● 主存中的一个存储单元所能存储的二进制位数

请同学们注意：

存储字长与机器字长不一定相同，例如机器字长为32位的计算机，所采用的存储字长可以是16位、32位或64位。



二、地址访问模式

主存中数据的存放 —— 地址访问模式

■ 主存通常按字节进行编址，而存储字长（主存中的一个存储单元所能存储的二进制位数）是字节的2的整数次幂倍（例如1字节，2字节，4字节）。

■ 以机器字长为32位的计算机为例，对主存的访问既可以按字节访问，也可以按16位半字访问，还可以按32位字访问。因此，可将主存地址分为：

- ☐ 字节地址
- ☐ 半字地址
- ☐ 字地址

【举例】

主存的存储体

	字节编址	字节地址	半字地址	字地址
xx	1011	1011	0101	0010
xx	1010	1010		
xx	1001	1001	0100	
xx	1000	1000		
AB	0111	0111	0011	0001
CD	0110	0110		
EF	0101	0101	0010	
00	0100	0100		
xx	0011	0011	0001	0000
xx	0010	0010		
xx	0001	0001	0000	
xx	0000	0000		

注:

- 字节地址逻辑右移一位，得到半字地址
- 半字地址逻辑右移一位，得到字地址

举例

主存中数据的存放 —— 地址访问模式

【举例】



Intel x86汇编语言访问不同存储单元举例
（假设数据段寄存器DS的值为0）

```
MOV AL, [0x4]   # 将地址0x4开始的1字节存储单元的内容送入
                # 8位寄存器AL，结果为[AL]=0x00

MOV AX, [0x4]   # 将地址0x4开始的2字节存储单元的内容送入
                # 16位寄存器AX，结果为[AX]=0xEF00

MOV EAX, [0x4]  # 将地址0x4开始的4字节存储单元的内容送入
                # 32位寄存器EAX，结果为[EAX]=0xABCDEF00
```

从上述程序可以看出，不同的地址访问模式（字节访问、半字访问、字访问）所使用的主存地址实际上都是字节地址，CPU在执行指令的时候可以将字节地址的低2位用于访问控制：

- 采用*字节访问模式，字节地址的低2位*用于选择字存储单元中的哪一个字节。
- 采用*半字访问模式，字节地址的倒数第2位*用于选择字存储单元中的哪个半字。

注:

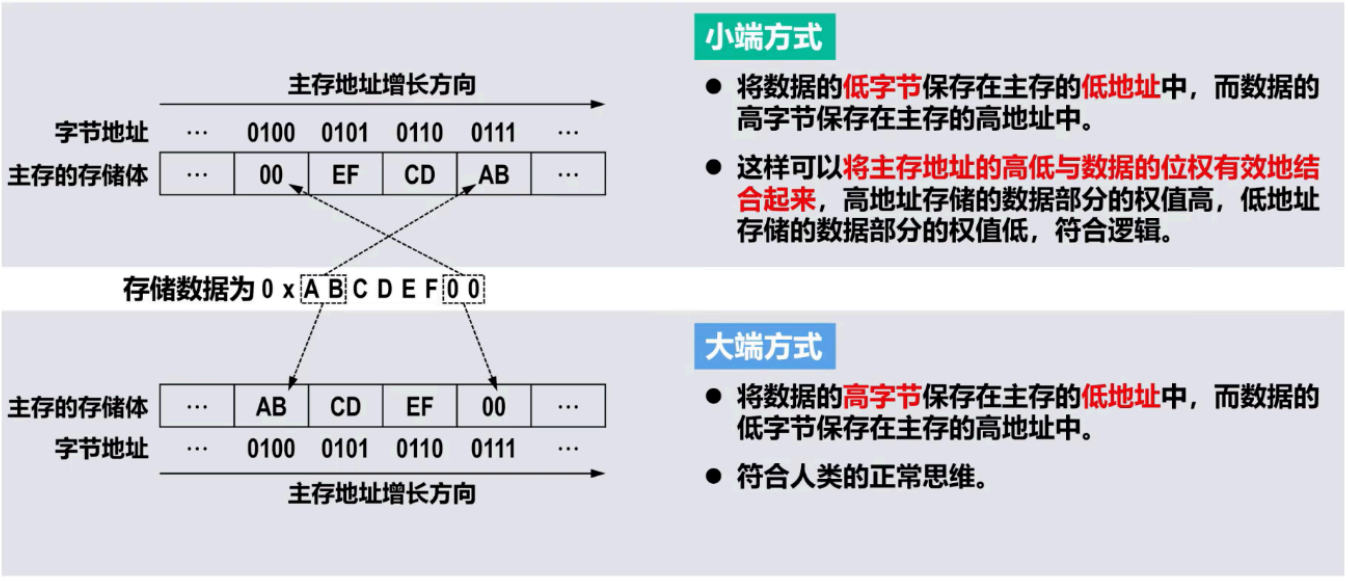
关于右下角的解释：以字为单元，即4字节时，有

- 若以字节（1字节）访问模式，由于存储单元是4字节，也就是4种可能，所以低二位组合，才能组成4种
- 若以半字（2字节）访问模式，由于存储单元是4字节，也就是2种可能，所以用倒数第二位来确定（倒数第一位不行，因为相邻的两个单元，倒数第一位必然不同，我们需要保证相邻）

三、大端和小端

举例

主存中数据的存放 —— 大端和小端方式



小端方式

- 将数据的低字节保存在主存的低地址中，而数据的高字节保存在主存的高地址中。
- 这样可以将主存地址的高低与数据的位权有效地结合起来，高地址存储的数据部分的权值高，低地址存储的数据部分的权值低，符合逻辑。

大端方式

- 将数据的高字节保存在主存的低地址中，而数据的低字节保存在主存的高地址中。
- 符合人类的正常思维。

主存中数据的存放 —— 大端和小端方式

小端方式	大端方式
● 将数据的低字节保存在主存的低地址中，而数据的高字节保存在主存的高地址中。	● 将数据的高字节保存在主存的低地址中，而数据的低字节保存在主存的高地址中。
● 这样可以将主存地址的高低与数据的位权有效地结合起来，高地址存储的数据部分的权值高，低地址存储的数据部分的权值低，符合逻辑。	● 符合人类的正常思维。
● Intel x86、IA64、RISC-V等处理器采用小端方式。	● PowerPC处理器采用大端方式；ARM、MIPS等处理器同时支持大端方式和小端方式。

请同学们注意：

1. 上述两种方式并没有绝对的优劣之分，它们在不同的处理器架构和应用场景中都有各自的适用性和优势。
2. 除处理器外，大小端方式还涉及外部设备设计、网络数据传输、音视频文件保存等。
3. 小端与大端方式的区别不仅存在于处理器的寄存器、存储器中，在指令集、系统总线等各个层次中也可能存在差别。

四、数据的边界对齐

4.1 数据对齐的原因

主存中数据的存放 —— 数据的边界对齐

■ 主存空间通常按字节进行编址。

■ 高级语言中不同数据类型的变量所包含的字节数量可能不同。

- ☐ 编译器在为这些变量分配主存空间时，理论上可以从主存空间的任何一个字节地址开始。
- ☐ 当一个多字节变量被编译器分布在不同的字存储单元中时，访问该变量就需要多个存取周期。
- ☐ 为了提高数据访问效率，应该要考虑数据变量、数据结构在主存空间中的边界对齐问题。

举例

主存中数据的存放 —— 数据的边界对齐

【举例】 假设存储字长为32位，为C语言不同数据类型的变量分配主存空间。

数据的边界*未对齐*

- 对存储空间的利用率最高
- 存在访问性能问题，例如：
  - 变量c的8个字节分布在3个存储单元中（访问该变量需要3个存取周期）
  - 变量e的2个字节分布在2个存储单元中（访问该变量需要2个存取周期）



数据的边界*对齐*

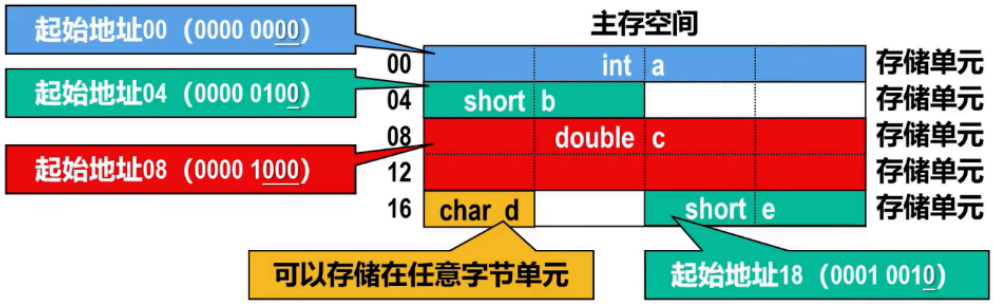
- 有效提升了访问性能，例如：
  - 变量c的8个字节分布在2个存储单元中（访问该变量需要2个存取周期）
  - 变量e的2个字节分布在1个存储单元中（访问该变量需要1个存取周期）
- 造成存储空间的浪费



4.2 边界对齐的规则

主存中数据的存放 —— 数据的边界对齐

【举例】 假设存储字长为32位，为C语言不同数据类型的变量分配主存空间。



边界对齐的规则

- 字节数据不存在边界对齐问题（因为主存空间就是按字节编址的）
- 半字（2字节）数据的起始字节地址的最低1位为0（即地址是2的整数倍）
- 单字（4字节）数据的起始字节地址的最低2位为00（即地址是4的整数倍）
- 双字（8字节）数据的起始字节地址的最低3位为000（即地址是8的整数倍）